

北京邮电大学
BEIDING UNIVERSITY OF POSTS AND TELECOMMUNICATIONS

第九章 纠删码

李丽香, 彭海朋
北京邮电大学网络空间安全学院
网络与交换技术国家重点实验室

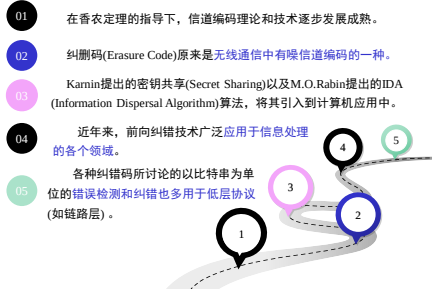


Wikipedia: In coding theory, an erasure code is **forward error correction** (FEC) code under the assumption of bit erasures (rather than bit errors), which transforms a message of k symbols into a longer message (code word) with n symbols such that the original message can be recovered from a subset of the n symbols.

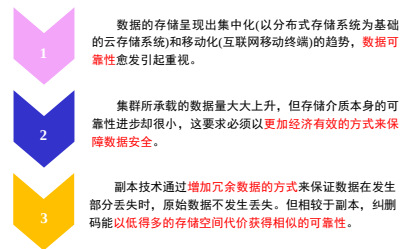


在编码理论中, 有一种前向纠错技术(FEC)也称为纠删码。
这种技术可以将原始数据中丢失的 k 字节数据从 n 个含编码字节的
信息中恢复。

Page 305



Page 306



Page 307

要想知道 x_1, x_2, x_3 的值, 只需要任意3个方程式即可解出。

$$\begin{aligned} x_1 &= 1 & (1) \\ x_2 &= 2 & (2) \\ x_3 &= 3 & (3) \\ x_1 + x_2 + x_3 &= 6 & (4) \\ x_1 + 2x_2 + 4x_3 &= 17 & (5) \\ x_1 + 3x_2 + 9x_3 &= 34 & (6) \end{aligned}$$

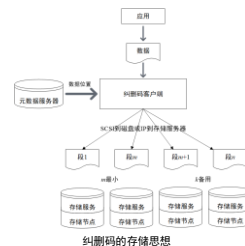
副本策略: 若方程(1)(2)(3)丢失任何一个方程数据都能恢复, 则需要把(1)(2)(3)三个方程都存储两份, 也就是存储6个方程。

纠删码策略: 把方程(1)(2)(3)看做是分布式系统的数据, (4)(5)(6)看做是code, 只要一个code, 即使丢了(1)(2)(3)中的任何一个方程, 数据都可以恢复, 故只需要存储4个方程。

策略存储效率对比: 纠删码策略存储4个方程, 副本策略存储6个方程, 存储效率是4/6, 可以节省1/3的空间。实际根据code的多少, 存储效率会不一样。

Page 308

纠删码是一组数据冗余和恢复算法的统称, 是一种数据保护方法。



纠删码技术将数据分割成片段, 把冗余数据块扩展、编码, 并将其存储在不同的位置, 比如磁盘、存储节点或者其它地理位置。

纠删码技术具有良好的容错性和安全性, 主要应用于网络传输中丢包错误的解决。

纠删码的存储思想

Page 309

网络传输方面：纠错码的基本思想是“当发送方将n个源数据包编码，得到m个校验数据包，将这n+m个数据包在网络上进行传输。接受方只要接收到不少于n个编码包，就可以将源数据恢复。一般来说，这个传输过程允许最多m个数据包的丢失。”



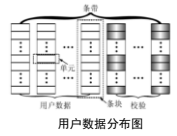
数据存储方面：纠错码的基本思想是“当冗余级别为n+m时，将这些数据包分别存放在n+m个硬盘上，这样就能容忍m个(假设初始数据有n个)硬盘发生故障。当不超过m个硬盘发生故障时，只需任意选取n个正常的数据块就能计算得到所有的原始数据。”



单元(Element)是用户数据或校验的一个基本单位，可以是一个比特(Bit)、一个字节(Byte)、一个扇区(Sector)或一个更大的磁盘数据块。它是纠错码的构造块(Building Block)。在编码理论中，它对应于码符号(Code Symbol)中的一个比特。

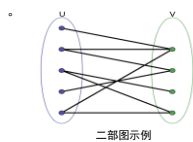
条带(SStripe)是一个由用户数据单元和校验单元组成的具有独立编码关系的集合。在编码理论中，它对应于一个码字(Code Word)。这里，条带长度(Stripe Length)定义为一个条带所跨越的磁盘个数，记为n。

条块(strip)是一个条带中存储在同一个磁盘上的所有单元组成的集合。在编码理论中，它对应于码字中的一个码符号。这里，条块宽度(strip Width)定义为一个条块中包含的单元个数，记为w。



Page 331

二部图又称为二分图，是图论中的一种特殊模型。设 $G=(V,E)$ 是一个无向图，如果顶点V可以分割为两个互不相交的子集(U,V)，并且图中每条边(i,j)所关联的两个顶点i和j分别属于这两个不同的顶点集(i in U, j in V)，则称图G是一个二部图。



二部图示例

范德蒙矩阵是法国数学家范德蒙提出的一种各列为几何级数的矩阵。

该矩阵第i行、第j列可以表示为 g_{ij}^{j-1} 。

$$G = \begin{bmatrix} g_0^0 & g_1^0 & g_2^0 & \dots & g_{n-1}^0 \\ g_0^1 & g_1^1 & g_2^1 & \dots & g_{n-1}^1 \\ g_0^2 & g_1^2 & g_2^2 & \dots & g_{n-1}^2 \\ \vdots & \vdots & \vdots & \dots & \vdots \\ g_0^{n-1} & g_1^{n-1} & g_2^{n-1} & \dots & g_{n-1}^{n-1} \end{bmatrix} = \begin{bmatrix} 1 & g_1 & g_1^2 & \dots & g_1^{n-1} \\ 1 & g_2 & g_2^2 & \dots & g_2^{n-1} \\ 1 & g_3 & g_3^2 & \dots & g_3^{n-1} \\ \vdots & \vdots & \vdots & \dots & \vdots \\ 1 & g_n & g_n^2 & \dots & g_n^{n-1} \end{bmatrix}$$

柯西矩阵中 $\{x_1, x_2, \dots, x_m\}$ 和 $\{y_1, y_2, \dots, y_m\}$ 为同一个有限域的两个不同子集。它们必须满足：

- (1) 不同子集中的任意两个元素相加不能为0。
 - (2) 这两个子集中的每个元素都不相同。
- 这两个条件都满足才能构成柯西矩阵。

Page 332

迦罗华域，用 $GF(2^n)$ 来表示，代表含有 2^n 个元素的有限域。迦罗华域中的四则运算(即基于多项式的运算)，多项式一般为 $f(x) = x^6 + x^4 + x^2 + x + 1$ 。

(1) 多项式加法

将两个多项式中相同指数的项系数相加或相减。例如 $(x^2 + x) + (x + 1) = x^2 + 2x + 1$

(2) 多项式乘法

将其中一个多项式的各项分别与另一个多项式的各项相乘，然后把相同指数的项的系数相加。例如 $(x^2 + x)(x + 1) = x^3 + x^2 + x^2 + x = x^3 + 2x^2 + x$

(3) 多项式除法

使用长除法。例如计算 $x^3 - 12x^2 - 42$ ，除以 $x^2 - 9x + 27$ ，商 $x - 27$ ，余数 -123 。

(4) $GF(2^n)$ 上的多项式运算

对于 $GF(2^n)$ 上的多项式计算，多项式系数只能取0或1。(如果是 $GF(3^n)$ ，那么系数可以取0、1、2)

$GF(2^n)$ 的多项式加法中，合并阶数相同的同类项时，由于 $0+0=0$ ， $1+1=0$ ， $0+1=1+0=1$ ，因此系数不是进行加法操作，而是进行异或操作。

$GF(2^n)$ 的多项式减法等于加法，例如 $x^4 - x^4$ 就等于 $x^4 + x^4$ 。

Page 333

编码方式：

将原数据重复k次后，就得到k-重复码。例如，将原数据x重复3次，拼接得到数据 $\{x, x, x\}$ ，即为原数据的3-重复码。发送方传输的就是k-重复码。

解码方式：

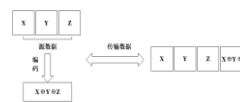
将接收到的数据分为k段，k为原数据的重复次数。每一段即是一个原数据，这样就完成了数据恢复。将这k段数据比较，可保证传输数据的可靠性。

Page 334

编码方式：

奇偶校验码是一种通过增加冗余位，使得码字中“1”的个数恒为奇数或者偶数的编码方法。

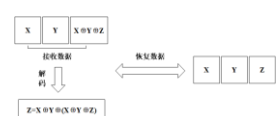
将原数据分为k个等长的部分，将这k个部分进行异或操作，与数据段尾部的冗余比较，以保证数据传输的可靠性。



奇偶校验码编码示例

解码方式：

冗余检测方式，接收方根据接收到的数据和冗余恢复出丢失的数据。也可以对接收到的数据进行异或操作，与数据段尾部的冗余比较，以保证数据传输的可靠性。



奇偶校验码解码示例

Page 335

重复码可容忍擦除率很高，但编码率低，效率低下，造成浪费。

奇偶校验码的编码率高，但可容忍擦除数少，容错性差。

二者的具体区别如表所示。其中n为原数据个数，m为冗余数。

	重复码	奇偶校验码
可容忍擦除数	最多 $(k-1)$ 倍于原数据包	只能容忍1个数据包
编码率	低，只有 $1/n$	高达 $n'/(m+n)$

Page 336

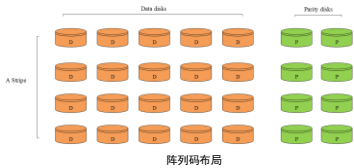
目前，纠错码技术在分布式存储系统中的应用主要有四类。



Page 337

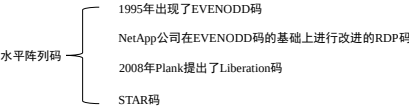
阵列码(Array Code)，即奇偶校验阵列码(Parity Array Code)的简称。它将一个条带中的用户数据单元和校验单元排列成一个二维或多维阵列，并基于XOR运算对条带内的单元进行编码。阵列码的提出主要是为了满足高容错磁盘阵列设计的需要。

阵列码分为水平阵列码、垂直阵列码和水平垂直阵列码。

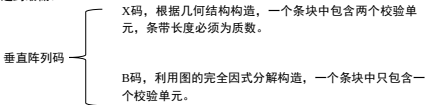


Page 338

水平阵列码的一个条带中的用户数据单元和校验单元分别存放在不同的条带中。



垂直阵列码的一个条带中的每个条带既包括用户数据单元，又包括校验单元。垂直阵列码的出现是为了使更新复杂度达到最低。



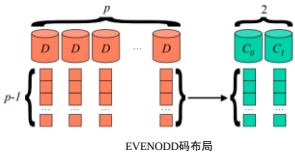
Page 339

水平垂直阵列码的一个条带中一些条带只包括校验单元，另一些条带同时包括用户数据单元和校验单元。

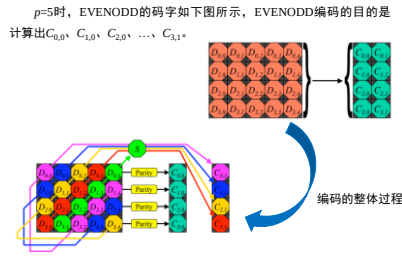


Page 340

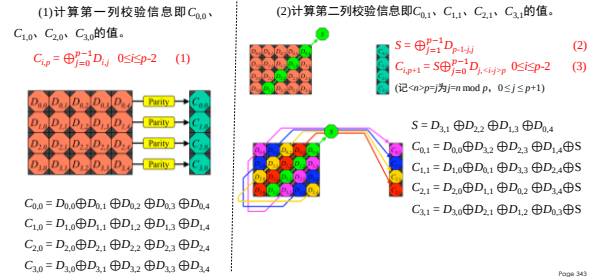
在阵列码中，EVENODD码是首个可以容忍两列丢失的编码方法，被公认为阵列码的始祖。在EVENODD码中，一共有p列数据列和2列冗余校验列，每一列有p-1个信息位，因此其码字为 $(p-1)*(p+2)$ 的二维阵列，其中p为大于2的素数。



Page 341



Page 342



Page 343

在这里介绍的是EVENODD双列删错的译码算法, 在译码过程中只需要简单的异或操作。
假定数据块 i 和 j 损坏, $0 \leq i < j \leq p-1$, 假设数据块的排列从左到右是 $0 \sim p-1$, 其中 $0 \sim p-1$ 列为用户数据块, $p, p+1$ 列为校验块。

(1) $i=p, j=p+1$, 两个校验数据块损坏, 数据恢复方式是重新编码。

(2) $i < p, j=p+1$, 一个用户数据块和一个校验块损坏, 恢复用户数据块 i 是通过参数 S 进行恢复, 即

$$S = D_{i,i+1+p,p+1} \oplus (\bigoplus_{j=0}^{p-1} D_{i,j+1+p,j}) \quad (4)$$

假定 $D_{p-1,j}=0, 0 \leq j \leq p$, 然后利用下式计算 i 数据块的信息:

$$D_{i,i} = S \oplus D_{i,i+1+p,p+1} \oplus (\bigoplus_{j=0, j \neq i}^{p-1} D_{i,j+1+p,j}), 0 \leq i \leq p-2 \quad (5)$$

Page 344

(3) $i < p, j=p$, 一个用户数据块和一个校验块损坏, 用户数据块 i 可以根据平行校验数据块进行异或得到恢复, 即

$$D_{i,i} = \bigoplus_{j=0, j \neq i}^p D_{i,j}, 0 \leq i \leq p-2 \quad (6)$$

(4) $i < p, j < p$, 两个用户数据块损坏, 数据恢复方法如下:

① 首先将最后两列校验数据块的信息异或, 恢复参数 S

$$S = (\bigoplus_{j=0}^{p-2} D_{i,j}) \oplus (\bigoplus_{j=0}^{p-2} D_{i,j+1}) \quad (7)$$

② 然后计算 $S^{(0)}=S_0^{(0)}, S_1^{(0)}, \dots, S_{p-1}^{(0)}, S^{(1)}=S_0^{(1)}, S_1^{(1)}, \dots, S_{p-1}^{(1)}$, 其中

$$S_0^{(0)} = \bigoplus_{j=0, j \neq i, j}^{p-1} D_{i,j} \quad (8)$$

$$S_0^{(1)} = S \oplus D_{i,p-1} \oplus (\bigoplus_{j=0, j \neq i, j}^{p-1} D_{i,j+1+p,j}) \quad (9)$$

③ 然后按照下面的顺序恢复 i, j 数据块的各个信息:

Page 345

- 1 Set $s \leftarrow \langle (j-i)-1 \rangle p$ and $a_{m-1,i} \leftarrow 0$ for $0 \leq i \leq p-1$
- 2 Let $a_{i,j} \leftarrow S_{\langle j+i \rangle p}^{(1)} \oplus a_{\langle s+(j-i)-p \rangle, p}$ and $a_{i,i} \leftarrow S_i^{(0)} \oplus a_{i,j}$
- 3 Set $s \leftarrow \langle s-(j-i)-1 \rangle p$ if $s=p-1$ then stop, else goto step 2

利用译码算法恢复第0和第2个错误数据块的信息:



初始数据						
?	0	?	1	0	1	1
?	1	?	0	0	0	1
?	1	?	0	0	1	1
?	1	?	1	1	0	0

Page 346

初始数据

?	0	?	1	0	1	1
?	1	?	0	0	0	1
?	1	?	0	0	1	1
?	1	?	1	1	0	0

第一步: 找到公共因子 S , 将这个数组的最后两列相异或, 得到 $S=1$ 。

这意味着, 对角校验是奇校验。根据公式(6)得到校验数组:

$$S^{(0)} = 01010, S^{(1)} = 01010$$

Page 347

第二步：利用递归恢复丢失的数据 $a_{i,j}$, $a_{i,2}$, $0 \leq i \leq 3$ 。

由于 $i = 0, j = 2$, 故 $s = \langle -(j-i)-1 \rangle p = \langle -3 \rangle 5 = 2$, 后续计算过程如表所示。

1	2	3	4
$S \leftarrow 2$	$a_{2,2} \leftarrow SD_2 \oplus 0$	$a_{2,2} \leftarrow SH_2 \oplus a_{2,2} = 0$	$S \leftarrow 0$
$S \leftarrow 0$	$a_{0,2} \leftarrow SD_2 \oplus a_{2,0} = 0$	$a_{0,2} \leftarrow SH_2 \oplus a_{0,2} = 0$	$S \leftarrow 3$
$S \leftarrow 3$	$a_{1,2} \leftarrow SD_2 \oplus a_{0,0} = 0$	$a_{1,0} \leftarrow SH_2 \oplus a_{1,2} = 1$	$S \leftarrow 1$
$S \leftarrow 1$	$a_{1,2} \leftarrow SD_2 \oplus a_{1,0} = 0$	$a_{1,0} \leftarrow SH_2 \oplus a_{1,2} = 1$	$S \leftarrow 4$

由于 $S=4=p-1$,故算法停止。

Page 348

定理一：EVENODD译码算法可以恢复任意2个数据块出错的数据。

证明：如果只有一列出错，我们可以利用剩余的正确校验位数据相异或进行恢复。根据异或的性质可以很容易知道这是完全可行的。

如果有两列出错，假设第 i 和 j 列出错， $0 \leq i < j \leq p-1$ 。我们假设译码算法的四种情况：

- (1) $i=p$, $j=p+1$, 这种情形和编码过程完全一样，显然此算法可行；
- (2) $i < p$, $j=p+1$, 注意公共因子 S 的校验值是由公式(4)得到的，它由对角线位 $\langle -i-1 \rangle p, 0, \langle -i-2 \rangle p, 1, \dots, \langle i, p-1 \rangle$ 上的数值相异或得到。这是一条唯一不与第 j 列相交的对角线(此时第 j 列无效)。因此所有的对角线都有相同的公共因子 S 。

Page 349

经过分析公式(3)得到 $a_{i,j}$ 的过程，我们可以推出能够找到恢复第 i 列数值的公式(5)，一旦第 i 列的值被恢复，这个算法就可以利用公式(1)得到第 p 列的值。

(3) $i < p$, $j=p$, 这种情形和编码的过程相似，在这里不再赘述。

(4) $i < j < p$, 第一步通过公式(7)计算出共同因子 S ，也就是公式(3)和公式(7)是等价的，但是公式(3)中存在未知的变量 i, j 的值，因此不能使用公式(3)直接得到共同因子 S ，因此根据公式(1)和(2)可以得到下面的式子(也就是公式(7))：

$$\begin{aligned}
 & (\oplus_{t=0}^{p-2} (\oplus_{l=0}^{p-2} a_{l,p+1})) = (\oplus_{t=0}^{p-2} (\oplus_{l=0}^{p-1} a_{l,j})) \oplus (\oplus_{t=0}^{p-2} (S \oplus \oplus_{l=0}^{p-1} a_{-i-l,p,j})) \\
 & = (\oplus_{t=0}^{p-2} (\oplus_{l=0}^{p-1} a_{l,j})) \oplus (p-1)S \oplus (\oplus_{t=0}^{p-2} (S \oplus \oplus_{l=0}^{p-1} a_{-i-l,p,j})) \\
 & = (\oplus_{t=0}^{p-2} (\oplus_{l=0}^{p-1} a_{l,j})) \oplus (\oplus_{t=0}^{p-2} (\oplus_{l=0}^{p-1} a_{-i-l,p,j})) \quad (10)
 \end{aligned}$$

Page 350

因为 $p-1$ 为偶数，所以 $(p-1)S=0 \bmod 2$ ，又因为最后一个虚拟的 $p-1$ 行所有的值都为0，即 $a_{p-1,j}=0$ ，因此可以得到 $(\oplus_{t=0}^{p-2} (\oplus_{l=0}^{p-1} a_{l,j}))$ 所有水平校验位的值异或，也就是所有的0到 $p-1$ 列的数据(每列是 $p-1$ 行)，所有数据位异或。 $(\oplus_{t=0}^{p-2} (\oplus_{l=0}^{p-1} a_{-i-l,p,j}))$ 表示所有的垂直校验位的值异或(除掉共同因子 S)，从0到 $p-1$ (每列 p 行，但最后一行的值为0)，因此也就是所有的0到 $p-1$ 列所有数据位异或。

因此：

$$\begin{aligned}
 0 & = (\oplus_{t=0}^{p-2} (\oplus_{l=0}^{p-1} a_{l,j})) \oplus (\oplus_{t=0}^{p-1} (\oplus_{l=0}^{p-1} a_{-i-l,p,j})) \\
 & = (\oplus_{t=0}^{p-2} (\oplus_{l=0}^{p-1} a_{l,j})) \oplus (\oplus_{t=0}^{p-2} (\oplus_{l=0}^{p-1} a_{-i-l,p,j})) \oplus (\oplus_{t=1}^{p-1} a_{p-1-l,j})
 \end{aligned}$$

所以得到如下公式：

$$(\oplus_{t=0}^{p-2} (\oplus_{l=0}^{p-1} a_{l,j})) \oplus (\oplus_{t=0}^{p-2} (\oplus_{l=0}^{p-1} a_{-i-l,p,j})) = \oplus_{t=1}^{p-1} a_{p-1-l,j} \quad (11)$$

公式(11)证明了公式(2)和公式(7)都能给出公共因子 S 的值。

Page 351

下面给出证明“这个算法只能唯一恢复第 i 列和第 j 列的数据”。

假设校验组 $S^{(0)}$ 和对角校验组 $S^{(1)}$ 已经分别由公式(8)和公式(9)获得。

第一步给 S 赋初值 $S = \langle -(j-i)-1 \rangle p$ 得到 S 。

第二步计算：

$$a_{\langle -(j-i)-1 \rangle p, j} \leftarrow S_{\langle -(j-i)-1 \rangle p}^{(1)} \quad (12)$$

该公式是正确的，通过公式(3)和公式(9)可以得出。

如果首先计算出 $a_{\langle -(j-i)-1 \rangle p, j}$ 的值，可以根据下面的公式(13)计算得到 $a_{\langle -(j-i)-1 \rangle p, j}$ ：

$$a_{\langle -(j-i)-1 \rangle p, j} \leftarrow S_{\langle -(j-i)-1 \rangle p}^{(0)} \oplus a_{\langle -(j-i)-1 \rangle p, j} \quad (13)$$

公式(13)的正确性，根据公式(1)和公式(8)可以得到。

第三步，参数 S 的设置为 $\langle -2(j-i)-1 \rangle p$ ，返回第二步，根据算法可以得到：

$$a_{\langle -2(j-i)-1 \rangle p, j} \leftarrow S_{\langle -2(j-i)-1 \rangle p}^{(1)} \oplus a_{\langle -(j-i)-1 \rangle p, j} \quad (14)$$

$$a_{\langle -2(j-i)-1 \rangle p, j} \leftarrow S_{\langle -2(j-i)-1 \rangle p}^{(0)} \oplus a_{\langle -2(j-i)-1 \rangle p, j} \quad (15)$$

Page 352

奇偶校验码，是一种完全基于XOR运算的纠错码，他是一种条块宽度为 $w-1$ 的特殊水平码。最简单的奇偶校验码是单奇偶校验码：在一个条带中，只有唯一的一个校验条块，这个校验条块由所有用户数据条块的XOR运算之和得到。于是单奇偶校验的容错能力为1，其最典型的应用是RAID 5。

基于单奇偶校验码可以构造出容错能力更高的奇偶校验码，大致可分为两类：

- (1)根据几何结构构造的奇偶校验码，例如水平垂直奇偶校验码
- (2)根据图结构构造的奇偶校验码，例如基于Tanner图的LDPC码

Page 353

根据几何结构构造的奇偶校验码。这类奇偶校验码将一个条带中的条块在逻辑上组织成多维几何结构，然后在每个方向上采用单奇偶校验码进行编码，因此具有十分规整的结构，十分易于实现。但是，他们的存储效率通常较低。

eg: 水平垂直奇偶校验码

编码方式：他将一个条带中的条块在逻辑上组织成二维阵列结构，然后在水平和垂直方向上分别采取单奇偶校验码进行编码。

$D_{0,0}$	$D_{0,1}$	...	$D_{0,(K-1)}$	$\rightarrow$	$P_{0,K}$
$D_{1,0}$	$D_{1,1}$	...	$D_{1,(K-1)}$	$\rightarrow$	$P_{1,K}$
$\vdots$	$\vdots$	$\ddots$	$\vdots$	$\rightarrow$	$\vdots$
$D_{(K-1),0}$	$D_{(K-1),1}$	...	$D_{(K-1),(K-1)}$	$\rightarrow$	$P_{(K-1),K}$
$\downarrow$	$\downarrow$	...	$\downarrow$	$\searrow$	$\downarrow$
$P_{0,K}$	$P_{1,K}$	...	$P_{(K-1),K}$		$P_{K,K}$

D_{ij} 表示用户数据条块

$P_{i,K}$ 表示校验条块

$$P_{i,K} = \sum_{j=0}^{K-1} D_{i,j}$$

$$P_{K,K} = \sum_{i=0}^{K-1} P_{i,K}$$

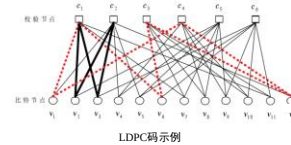
$$P_{K,K} = \sum_{i=0}^{K-1} \sum_{j=0}^{K-1} D_{i,j}$$

$$(0 \leq i \leq K-1, 0 \leq j \leq K-1)$$

Page 354

低密度奇偶校验码(Low-Density Parity-Check Code, LDPC)最初是为通信链路的容错而设计的，近年来开始基于广域网的存储应用中被研究。LDPC码的主要优点在于，构造出的高容错LDPC码可以具有接近最高的存储效率并且解码复杂度很低。

LDPC码是一种由级联的随机稀疏二部图和一个传统的纠错码构成的特殊的纠错码，级联二部图的意思就是多级二部图。



LDPC码示例

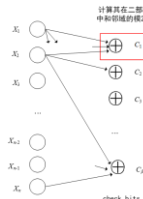
Page 355

低密度奇偶校验码的一个典型的例子是龙卷风码(Tornado Code)

编码方式：

龙卷风码通过层层递进的异或操作得到校验码。

如图所示，把原数据分成n个等长的数据块($x_1, x_2, x_3, \dots, x_n$)，这些数据被称为用户数据块(message bits)。根据多级二部图 $B_0, B_1, \dots, B_{k-1}$ 的连接关系，将相连的数据节点进行异或得到下一层校验节点(check bits)的值($C_1, C_2, \dots, C_p$)。



龙卷风码编码示例

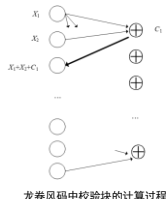
Page 356

举例来说，在右图中，某校验层各节点的值分别为 $x_1, x_2, x_3, \dots, x_n$ ，下一级第一个校验块由 $x_1 \oplus x_2$ 得到。

选择哪些变量来进行异或操作由二部图确定，一般来说，级联低密度纠错码二部图的生成具有随机性，通过使用每一级二部图的关系，就可以依次生成级联二部图的所有右节点。

在此过程中，进行异或操作的次数(或者说是运算层数)由每层中的数据块数目决定。

不妨引入一个参数 $e, e = m/(m+n)$ ，其中 m 是最终的校验码数目，也就是说， e 是擦除概率，它与码率的和为1。显然， e 满足 $e < 1$ 。那么，在校验块计算时，第一层得到的数据块数目就是 $e \times n$ ，第二层的数目是 $e^2 \times n$ ，第三层的数目是 $e^3 \times n$ ，依次递推，第 k 层的数据块数目就是 $e^k \times n$ 。

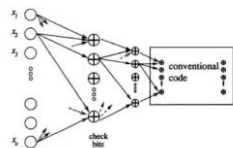


龙卷风码中校验块的计算过程

Page 357

下图说明，最后一级(也就是二部图中的 B_{k-1})的计算使用传统的编码方法(如RS码的编码方法)。最后一级的运算中， B_{k-1} 和 B_k 具有相同的变量节点，根据连接关系生成新节点。

至此，整个编码工作完成。发送方传输的编码就是原数据包加上校验块的总数数据包。



龙卷风码解码过程

Page 358

解码方式：

接收方收到 $n+m$ 个数据包，如果包含源数据的 n 个数据包有丢失，就可以通过 m 个冗余数据包进行重构。

解码过程实际上是一个在二部图上去边的过程，从最高级二部图开始：

- (1)首先，利用 B_k 和 B_{k-1} 得到它们的信息块，即 B_{k-1} 的校验块。
- (2)然后，依次得到 $B_{k-2}, \dots, B_1, B_0$ 的校验块。
- (3)最后，得到 B_0 的 m 个信息块。

当然，如果在解码开始时就已经得到了某一层 $B_i (0 \leq i < k)$ 的所有校验块，那么就可以直接从 B_i 开始解码。

Page 359

容错性:

如果与某一校验节点相邻的信息节点中存在一个丢失数据的信息节点,那么可以通过该校验节点和其它节点进行异或运算,得到丢失的信息数据。

数据恢复:

这里的“信息节点”与“校验节点”是相对的,在每一层异或运算中,参数节点称为“信息节点”,生成节点称为“校验节点”。具体步骤如下:

- (1)所有校验节点赋初值0,所有信息节点根据接收的信息赋值为0,1,或D (删除)。
- (2)根据边的连接关系,把所有未被删除的信息节点的值和与其相连的校验节点的值(模二)相加,并将得到的值作为该校验节点的新值,然后从二部图中去掉这些正确接收的变量节点以及与之相连的边。

(3)在得到的二部图中寻找度数为1的校验节点。如果某一校验节点度数为1,则该校验节点的值就等于唯一与它相连的信息节点的值,此信息节点所对应的信息比特得到恢复。

(4)再根据边的连接关系,把恢复出来的信息节点的值与和它相连的校验节点的值(模二)相加,作为这些校验节点的新值,并从二部图中去掉恢复出来的信息节点和与之相连的边。

(5)在去边之后的二部图中重复步骤(3)(4),直至二部图中不存在度数为1的校验节点,或所有的原数据(message bits)值均被恢复。

这样,所有能恢复的丢失数据都能被恢复。此解码过程中只有异或计算,故解码速度较其他纠错码快速。

龙卷风码的复杂度分析	Tornado code
编码时间复杂度	$O(n \ln(1/e))$
解码时间复杂度	$O(n \ln(1/e))$
编码率	$1 - p(e) \sim \exp(-1 + e)$

优点:

- (1)由于只涉及异或操作,编码率高,其中 $p(e)$ (e 是擦除概率)是一个极小量,与二部图的选择有关。
- (2)编码过程提到,进行异或操作的次数,也就是二部图的边的数目,由每层中的数据块数目决定。因此该算法的复杂度为线性,编码码效率高。
- (3)由于所有的丢失数据基本能恢复,龙卷风码能提供接近最优的损失保护。

缺点:

- (1)需要擦除概率 e 作为输入,而擦除概率难以精确获得、随时可能变化、多个接受者的擦除概率不同的特点也带来了这种算法的缺点。
- (2)和RS Code一样,在数据规模较大,且在数据规模较大的情况下,编解码优势才比较明显。数据规模较小时并不能体现很大优势。

数字喷泉码是一类码率不受限制的纠错码(Rateless Erasure Code),也称为无速率编码,是一类基于图的线性纠错码。喷泉码是指可以由 k 个原始数据分组生成任意数量的编码分组,而接收方只要收到其中任意 m 个编码分组,即可通过译码以高概率成功恢复全部原始数据分组。

可以看到,上述编码过程就如同源源不断产生水滴(编码分组)的喷泉(编码器),而我们只要用杯子(译码器)接收足够数量的水滴,即可达到饮用(成功译码)的目的。正因如此,该种编码被称为喷泉码。



数字喷泉码的形象说明



John Byers 及Michael Luby 等人于1998 年首次提出了数字喷泉(Digital Fountain)的概念,它是针对大规模数据分发和可靠广播的应用特点而提出的一种理想的解决方案。但当时并未给出实用数字喷泉码设计方案。



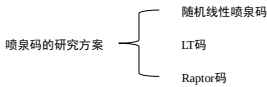
2002年, M.Luby提出了第一种现实可行的喷泉码——LT码。在学术理论日渐完善的同时,喷泉码也日益受到产业界的关注,获得了越来越多的实际应用。



目前,一种由Digital Fountain公司设计的系统Raptor码也已经 被DVB-H标准和3GPP组织的MBMS标准采用,并且正在参与其他 多项国际标准的制定。

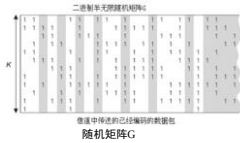
喷泉码的研究现状:

- (1)应该尽量减小译码开销 c ,使其趋于零。
- (2)应该尽量减小编译码复杂度,理想情况下,应该使生成每个编码分组需要的运算量是一个与 k 无关的常数,而成功译码 m 个编码分组获得 k 个原始数据分组需要的运算量是一个关于 k 的线性函数。

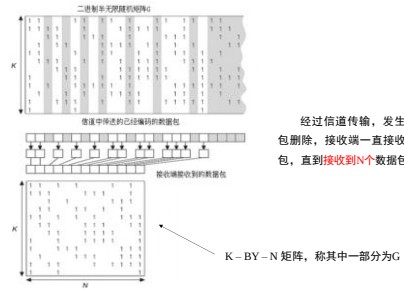


假设编码器需要编码的文件为 $S_1, S_2, \dots, S_k$ ，每个 S_i 都是一个信息符号，它只能被完整的传输或者被删除。假定每个 S_i 大小都是相同的比特数。在每一个时钟周期，编码器产生一个 k 阶随机二进制矩阵 G_{kn} ，编码器的输出由 S_i 异或而成，**参加运算的 S_i 对应的 $G_{kn}=1$** ，即 $t_n = \sum_{k=1}^n S_k G_{kn}$

这个和可连续的由不同的 S_i 异或而来，可以对**每 k 个随机 t_n 在一个半无限的二进制矩阵中定义一个新的列**，如图所示。



Page 364



Page 367

(1) 如果 $N < K$ ，接收端就没有接收到足够的码元，不能恢复源文件。

- 如果 $N = K$ ，则接收端有可能恢复源文件。如果矩阵 G 可逆（二进制），就可以计算出 G 的逆矩阵 G^{-1} ，就能恢复 S_i ，即 $S_k = \sum_{n=1}^k t_n G_{kn}^{-1}$ 。此时数据恢复的概率为 $(1-2^{-8})(1-2^{-(K-1)}) \dots (1-1/8)(1-1/4)(1-1/2)$ ，当 $K > 10$ 时，概率约为0.289。
- 如果 $N > K$ ，令 $N = K + E$ ， E 是多接收到的很小量的码元。令此时数据恢复的概率为 $1-e$ ， e 为接收端接收到额外的码元时无法恢复的概率。对于任何 K ，无法恢复源文件的概率 $\alpha(E) \leq 2^{-E}$ 。总的来说，恢复源文件的概率为 $1-e$ 时，需要接收的数据包数目大概是 $K + \log_2(1/e)$ ，对一个数据包编码的代价是需要 $K/2$ 次数据包运算，解码的代价是对随机矩阵 G 求逆的代价(大概 K^2 次二进制运算)和对接收的数据包对应的随机矩阵求逆的代价(大概是 $K^2/2$ 次数据包运算)之和。

当一个随机码对删除信道而言并不是技术上的最佳码，在接收端接收到 K 个数据包时，只有0.289的概率恢复源文件，这几乎可以认为是最好的了。但当接收到额外的 E 个数据包时，无差错的恢复源文件的概率就至少为 $1-e$ ，其中 $e=1-2^{-E}$ 。因此，随着源文件大小的增加，随机线性喷泉码能够无限地接近香农理论的极限。唯一的缺点是随机线性喷泉码的编码和解码的代价都是二次方，编码时还会出现三次方。当 K 比较小时，这并不重要，但若 K 很大时，编码和解码的代价就会很大。因此，我们希望更小的编码和解码代价来解决问题。

Page 368

将长为 n 的文件分割成 $k=n/w$ 个等长的输入符号 s_k (每个输入符号的长为 w)，每个数据包称为输入符号，每个编码包为输出符号。若一个输出符号结点的度为 d ，定义LT码的度分布 $p(d)$ ($d \geq 1$)为 d 的概率分布。编码步骤如下：

- 编码器首先在 $1-k$ 范围内按某一分布 p (称为编码度分布)**随机选取一个整数 d** ，其中 k 称为该码的码长， d 称为编码分组的度；
 - 在所有输入符号中随机均匀地选取 d 个符号**作为该编码符号的邻节点符号；
 - 将 d 个数据进行异或并输出**，即对这 d 个输入符号求异或得到一个编码符号 t_n 。
- 不断重复上面的步骤，就可以得到无限多个编码分组。

在这些步骤中一些细节需要注意：**LT码编码之前要先确定度分布函数**，度值的确定由度分布函数来计算。不同的度分布， k 的概率值是不同的，但**度分布概率函数只是确定了取度值为 d 的概率是多少，而不能确定这 d 个原始编码分组是哪些**，所以**对于这 d 个原始编码分组的选择来说是任意的**，这里对这 d 个原始编码分组的选择采用均匀分布来进行选取，即从区间 $[1, k]$ 中按均匀分布随机生成 d 个整数。区间 $[1, k]$ 中所有整数被选取的概率是相等的。把这 d 个整数进行异或，就得到了一个编码分组。

Page 369

随机度 d 的确定(弧波分布)：

$$p(d) = \begin{cases} \frac{1}{k} & d = 1 \\ \frac{1}{d(d-1)} & d = 2, 3, \dots, k \end{cases} \quad (1)$$

根据度分布函数确定编码时的度(参与编码的源文件的个数)。

假设 $k=100$ ，则由弧波分布可知：

度为1的概率为1/100，度为2的概率为1/2，...， d 为100的概率为1/9900。

在 $[1, 10000]$ 区间上划分100个区间：

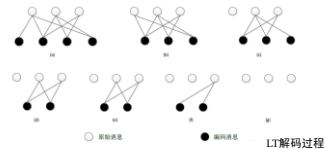
度	取该度的概率	取该度的空间	S_k	取该度的概率	取该度的空间
1	1/100	(0,100]	S_1	1/100	(0,100]
2	1/2	(100,5100]	S_2	1/100	(100,200]
3	1/6	(5100,6766]	S_3	1/100	(200,300]
...	...	...	...	...	...
100	1/9900	(9999,10000]	S_{100}	1/100	(9900,10000]

在0到10000之间随机产生 d 个数，看这 d 个数落在哪个区间，选取这些区间所对应的 s_k 进行编码。

Page 370

LT码的解码采取**迭代法(BP算法)**。在解码的每一步，解码器都在**编码符号集合中寻找度为1的符号，从而直接恢复出它们所连接的输入符号**，然后将这些输入符号与跟它们有连接的编码符号进行异或并将结果取代该编码符号原来的值，完成之后删去它们之间的连接关系，并相应地修改编码符号的度。重复上述过程直至不存在度为1的符号为止。

如果所有输入符号都被恢复则解码成功，否则解码失败。



Page 371

LT码的解码

具体步骤是：

(1)如图(a)所示,接收一定数量的编码分组,根据编码分组间的对应关系(可以通过分组头部等方式显式传递,也可以通过事先约定伪随机序列等方式隐式传递)建立双向图。顶层节点代表原始分组,底层节点代表编码分组,连接它们的边代表该原始分组是该编码分组的模二和分量。

(2)任意选取一个度数为1的编码分组。如果不存在,则译码停止;如果存在,则通过简单的复制运算,即可恢复与之相连的唯一原始分组,如图(b)所示。

(3) 将已经恢复的原始分组模二之和到与其相连的所有其他编码分组中, 消除其在这些编码分组中的模二和分量。相应地, 将双向图中对应的边删除, 使得这些编码分组的度数减1。如图 (c) 所示。如果某个编码分组的度数减小为1, 则称该编码分组被“释放”, 如图(c)中最右侧的编码分组。

(4)重复步骤(2)和(3),直至译码停止。如果所有原始分组都已经恢复,则译码成功;否则,译码失败,必须接收更多的编码分组才能继续译码。

一般来说,为了保证所有数据均能恢复出来,度的选择期望值必须高于 $O(\log n)$,舍弃编码过程中关于计算量的考虑。

LT码的特点

	LT码
编码时间复杂度	$O(n \log n)$
解码时间复杂度	$O(n \log n)$
编码率	低

LT码的编解码时间复杂度均为 $O(n \log n)$ 。另外, LT码因为随机性, 编码率并不确定, 不过确定的是其编码效率较为低下。

Raptor码

产生原因:

LT码生成的编码包中有少量连接度很高的包，这些**高连接度包的存在**消耗了很多编解码异或操作，同时也降低了低连接度包的比例，从而减小了译码过程中可译集的大小，**降低了译码成功率**。

基本思想:

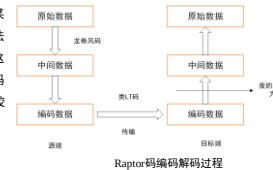
Raptor码提出采用两步编码的方式。首先对原始信息用一个分组码进行预编码，然后采用一个弱化的LT码对数据进行编码并发送。

课堂思政

Raptor码是在LT码的基础上发展出来的改进版，Raptor码的出现告诉我们有问题不可怕，改正就是了。正如党的十八大以来习总书记反复强调的“直面问题、解决问题”的思想，并且敢于直面问题、勇于修正错误使我们党的显著特点和优势。

Raptor码的编解码

如下图所示,通过**预编码**过程,首先采用一种传统的编码方法(图中采用龙卷风码编码方法),如同Raptor码的命名Rapid-Tornado所暗示,这里**一般采用的就是龙卷风码**,将原始数据编码成为中间编码校验单元。然后将这些中间编码校验单元作为LT码的输入符号,进行编码。



与编码过程相对应地, 解码时只需要利用LT码解码技术恢复所有的中间编码校验单元, 再根据中间编码的获得方式, 使用传统编码的解码方式得到源数据包。

Raptor码的特点



EC码的应用

替代RAID

RAID的目的是在硬盘驱动器(HDD)失效时保护数据，它里面的数据可通过奇偶校验或镜像副本重新创建，但电子机械式硬盘发生故障的几率很高。Erasure code 存储消除了所有RAID问题。2009年Google在第二代GFS中使用了RS(6,3)编码，Facebook早期也在HDFS中使用了RS(10,4)编码。

各大互联网公司的存储服务

例如Google在GFS中使用了RS(6,3)编码，Facebook在HDFS中使用了RS(10,4)编码，Microsoft在Azure中使用了LRC(12,2,2)编码，阿里云在盘古中使用了RS(8,3)编码。

应用在AWCloud中

AWCloud中利用FPGA技术提高了纠错码的性能。

- 从数据类型的角度而言，**归档数据适合使用纠删码**。首先，归档数据需要保存相当长的一段时间，一份、两份或多份数据出现问题的可能性会急剧上升；其次，归档数据的写入操作很少，使得纠删码可以很轻易的恢复数据，并且纠删码的额外开销很少。如果没有故障发生，纠删码对读取操作通常不会有任何影响。
- 从应用的角度来看，对于读写都很繁忙的应用，使用纠删码并不是一个很好的选择。但对于**只有大量读取的应用**，纠删码可以提供强大的可靠性，并且在恢复数据的时候只产生很小的额外开销，因为纠删码的额外开销主要来自写操作。

——来自TechTarget中国原创内容中Miller的回答

Page 378

- 评判纠删码的**第一条标准是性能**：即部署了一种纠删码后写入速度有多快，进行数据恢复时的读取速度有多快。
- 另一个标准是**纠删码如何将数据单元分割**：可以是整盘的分割，可以是卷级的分割，也可以是部分盘或部分卷的分割等等。**分割时还会遇到的一个问题是纠删码单元的数量有多少**，有10个数据单元和5个纠删码单元组成的纠删码系统，也有12个数据单元和4个纠删码单元组成的纠删码系统。但大部分纠删码产品都能配置数据单元和纠删码单元的个数。
- 最后需要考虑的是**某些纠删码类型会带来一些问题**：比如在部署之前需要了解每个厂商的纠删码是如何工作的，它们能在什么样的场景恢复数据故障。

——来自TechTarget中国原创内容中Miller的回答

Page 379

- 在某些领域，纠删码会在未来掀起一波浪潮，特别是在**存储归档领域纠删码是必备的**。因为当把数据保存10年甚至20年时会经历各种故障并将它们恢复，并且在数据归档中，需要经常读取数据，但不会产生额外的性能开销。
- 此外，未来纠删码是否会成为潮流**取决于设备容量大小和读写速率的相对速度提升差异**。当存储容量的增速大于读写速率的增速时，纠删码的作用就更能体现，这就是为什么从90年代中期的RAID5走到了今天的RAID6。

——来自TechTarget中国原创内容中Miller的回答

Page 380

灵活的设定副本数量，从而在数据持久性和成本间做出权衡，满足不同的应用场景。比如储存冷数据时，可将副本数缩小至1.2，即冗余数据是0.2。

Erasure code在**面对自然灾害或技术故障时具有很好的恢复能力**。另外，Erasure code允许同时发生多种故障。

Erasure code有**内置的数据安全性**。Erasure code使用的**数据块可以分布在城市、地区、一个数据中心或全球任何地方的不同存储位置**。

Page 381

纠删码技术提高存储资源利用率的表现□：提供近似三副本的可靠性

纠删码技术提高存储资源利用率的表现□：减小冗余存储□

纠删码技术的代价1：□□□计算量□

纠删码技术的代价□：网络负载□

EC码的优化——LRC

Page 382